

2026 嵌入式Linux

1. 驱动中对共享数据的保护

1. **原子保护**：一般原子操作用于变量或者位操作

（原子操作只能对整形变量或者位进行保护）

2. **自旋锁**：当一个线程要访问某个共享资源的时候首先要先获取相应的锁，锁只能被一个线程持有，只要此线程不释放持有的锁，那么其他的线程就不能获取此锁。**等待自旋锁的线程会一直处于自旋状态，自旋锁适用于短时期的轻量级加锁**

被自旋锁保护的临界区一定不能调用任何能够引起睡眠和阻塞的 API 函数，否则的话可能会导致死锁现象的发生

不能递归申请自旋锁，自旋锁会自动禁止抢占

3. **信号量**：使等待资源线程进入休眠状态，因此**适用于那些占用资源比较久的场合**

信号量会引起休眠，不能用于中断中

如果共享资源的持有时间比较短，不适合使用信号量（切换线程引起的开销大）

4. **互斥体**：互斥访问表示一次只有一个线程可以访问共享资源，不能递归申请互斥体

mutex可以导致休眠，因此不能在中断中使用mutex，中断中只能使用自旋锁

和信号量一样，mutex保护的临界区不可以调用引起阻塞的API函数

必须由mutex的持有者释放mutex。并且mutex不能递归上锁和解锁

2. Uboot启动流程

其启动流程是系统加电或复位时，**初始化硬件、加载内核并最终跳转到操作系统**的过程

1. **基本硬件初始化**：当系统上电或复位后，CPU开始从预定义的地址执行代码，用于初始化一些基本的硬件，例如时钟、引导存储设备（eMMC、NAND）

2. **SPL加载**：U-boot精简版，主要负责初始化更复杂的硬件（例如DDR内存控制器），并加载完整的U-boot映像到内存中。SPL通常位于存储器的前几个扇区

一旦内存初始化完成，SPL会从Flash、eMMC、SD卡等存储设备加载完整的Uboot映像到内存中

3. U-Boot启动

（1）**硬件初始化**：CPU和缓存的设置、串口初始化、外设总线和外设初始化、加载设备树

（2）**环境变量初始化**：从Flash或其他存储介质，这些变量用于定义系统的启动参数，如引导命令、内核路径等

解析环境变量中的启动命令（bootcmd，定义了如何加载操作系统的内核和启动参数）

4. 引导操作系统

选择启动设备：根据环境变量配置，选择合适的启动设备来加载操作系统的内核映像

内核加载：U-Boot会从存储设备中读取操作系统的内核映像和设备树，并将他们加载到指定的内存地址

传递参数：U-Boot会将启动参数（内核启动命令行和设备树）传递给操作系统内核

跳转到内核

3. 字符设备驱动框架

1. 在模块的入口函数中申请设备号（分配设备描述结构）

手动分配（register_chrdev_region）或者动态分配（alloc_chr_region）

（dev_t）主设备号区分设备的类型，次设备号标记同类型的不同设备

2. 定义、初始化、注册设备描述结构

定义cdev，构建设备操作集（file_operations），初始化（cdev_init）

添加字符设备到内核（cdev_add）

3. 创建设备节点

创建类（class_create）、创建设备（device_create）

使用mdev来实现设备节点文件的自动创建与删除（busybox创建）

4. 获取设备节点，硬件初始化

按路径获取设备节点（of_find_node_by_path），获取gpio属性

申请IO，设置IO（pinctrl：在设备树进行电气属性配置，gpio子系统：设置输入输出，输出高低电平）

5. 实现设备操作

实现设备操作指令集file_operations

open：进行硬件初始化，通常设置文件私有属性（设置filp的private_data 成员变量指向设备结构体）

write/read：需要 copy_to_user() 用于将内核数据传送给用户空间

copy_from_user() 用于将用户空间的数据传送给内核空间

poll：轮询函数，用于查询设备是否可以非阻塞的读写

unlocked_ioctl：提供对于设备的控制功能，与应用程序中的ioctl函数对应

fsync：刷新待处理的数据，用于将缓冲区中的数据刷新到磁盘中

(使用异步通知：调用fasync_helper初始化fasync_struct)

当应用程序通过“fcntl(fd, F_SETFL, flags | FASYNC)”改变 fasync标记的时候，驱动中fasync会执行

6.在模块的出口函数中注销驱动

删除设备device_destory

删除类class_destory

删除字符设备cdev_del

释放设备号unregister_chr_region

7.其他：添加LICENSE和作者信息

4. platform总线设备驱动/驱动和设备的匹配

1.总线 (bus_type) 主要完成总线下驱动和设备的匹配 (match)

(1) platform总线是虚拟总线，管理所有 platform 设备和驱动，当新设备 / 驱动注册时，触发 platform_match 函数进行匹配

(2) platform设备描述片上外设的**硬件资源**，嵌入式中主要通过**设备树 (DTS)** 定义

(3) platdorm驱动实现外设的**驱动逻辑**，核心是 probe 函数 (匹配成功后执行，初始化外设)

2.向总线注册驱动

向总线注册驱动时，先将驱动添加到总线下的驱动链表，总线的 match 函数检查当前总线下的所有设备，有没有与此驱动的 compatible 匹配 (of_match_table) 的设备，如果有的话就执行驱动里面的 probe函数

设备树匹配：内核启动时根据设备树自动产生的设备

匹配成功后可以在bus->driver下找到相应的设备节点，用于设备初始化

(1) 向总线注册驱动

driver_register 注册驱动

```
1 bus_add_driver 将驱动添加到总线下去
2 driver_attach 查找总线下所有与驱动匹配的设备
3 bus_for_each_dev
4 __driver_attch 查看每个设备是否与驱动匹配
5 driver_match_device 检查是否匹配
6 bus->match
7 driver_probe_device 匹配成功
8 really_probe
9 drv->probe 执行driver的probe函数
```

(2) 向总线注册设备

device_register 注册设备

```
1  device_add
2      bus_add_device 将设备添加到总线下去
3      bus_probe_device
4          device_attach
5              bus_for_each_driver
6                  __device_attach
7                      driver_match_device 匹配驱动
8                          bus->match
9                              driver_probe_device 执行此函数
10                                 really_probe
11                                     drv->probe 执行driver下的probe函数
```

3.platform_match匹配过程

(1) 当platform设备或驱动注册到总线时，内核会调用 `platform_match` 函数进行匹配
设备树匹配 (of_match_table) → ACPI匹配 → ID表匹配 (id_table) → 驱动名匹配 (driver.name)

(2) of_match_table

内核遍历驱动的 `of_match_table`，对比设备树节点的 `compatible` 字符串，完全一致则匹配成功

(3) probe 函数的核心工作：

- ① 通过 `platform_get_resource` 获取设备的寄存器地址、中断号等硬件资源；② 申请内存映射 (`ioremap`)，将物理地址转为虚拟地址；
- ③ 申请中断 (`request_irq`)，注册中断处理函数；
- ④ 初始化外设硬件（配置寄存器）；
- ⑤ 注册字符设备 (`cdev_add`)，向上层提供操作接口。

(4) 向内核注册platform驱动

platform_driver_register

```
1  platform_driver_register
2      设置driver的probe为platform_drv_probe
3      __platform_driver_register
4          driver_register
5              匹配后执行driver->probe，也就是platform_drv_probe，最终会调用
                platform_driver下的probe函数
```

(5) platform_match 驱动和设备匹配

platform_match 驱动和设备匹配

```
1  of_driver_match_device 使用设备树
2      of_match_device driver下的of_match_table (设备树匹配表, compatible属性值)
3      of_match_node
4      __of_match_node
5      __of_device_is_compatible
6      __of_find_property 取出compatible属性值
7  acpi_driver_match_device ACPI类型
8  platform_match_id 根据platform_driver下的id_table
9  strcmp(pdev->name, drv->name) 比较字符串name, 就是platform_device下的name和
    platform_driver->name (无设备树)
```

5. 阻塞、非阻塞和异步通知

阻塞IO是等待设备可访问后访问，**非阻塞IO**是查询设备是否可以访问，**异步通知**是设备通知自身可以访问

1.阻塞IO（IO指的是应用程序对驱动设备的输入/输出操作）：等待队列

如果不能获取到设备资源，那么阻塞式IO就会将应用程序对应的线程挂起（当设备文件不可操作的时候进程可以进入休眠态），直到设备资源可以获取为止

在驱动中使用等待队列

创建并初始化一个队列头

定义并初始化一个等待队列项（使用宏DECLARE_WAITQUEUE）

将队列项添加到等待队列头

等待唤醒：wake_up（主动唤醒） / 设置等待队列等待某个事件 wait_event

从等待队列头移除等待队列项

2.非阻塞IO：轮询

应用程序对应的线程不会挂起，会一直轮询等待，直到设备资源可以使用

(1) 应用程序通过select、epoll或poll函数来 查询设备是否可以操作

select：在单个线程中，select函数能够监视的文件描述符数量有最大的限制，一般为1024

poll：poll函数没有最大文件描述符限制

传统的selcet和poll函数都会随着所监听的fd数量的增加，出现效率低下的问题，而且 poll函数每次必须遍历所有的描述符来检查就绪的描述符

epoll：为处理大并发而准备的，一般常常在网络编程中使用epoll函数

(2) 可以操作的话会调用设备驱动程序中的poll函数

在驱动程序的poll函数中调用poll_wait函数，poll_wait函数不会引起阻塞，只是将应用程序添加到poll_table中

3.异步通知：信号

一旦设备就绪，主动通知应用程序。这样应用程序根本不需要查询设备状态

信号相当于软件层次上对中断的模拟，驱动可以通过主动向应用程序发送信号的方式报告可以访问

驱动中的信号处理：设置fasync_struct结构体（fasync_helper初始化），实现操作集中fasync函数

在需要通知时调用kill_fasync 函数负责发送指定的信号

应用程序对异步通知的处理

- 1，注册信号处理函数（使用signal函数来设置指定信号的处理函数）
- 2，将本应用程序的进程号告诉给内核（fcntl(fd, F_SETOWN, getpid())）
- 3，开启异步通知

通过fcntl函数设置进程状态为FASYNC，驱动中的fasync函数就会执行

6. 中断子系统

1.核心架构

硬件中断源->中断控制器（GIC）->内核中断核心层（irq/core.c）->中断处理框架（上下半部）->驱动层中断处理函数

中断号：内核为每个中断分配的唯一编号（非硬件中断线号）

ARM 架构中通过 `irq_of_parse_and_map`（设备树）或 `platform_get_irq`（platform 设备）获取

中断控制器：ARM 架构用 GIC，负责汇总外设中断、优先级管理、中断分发（替代 STM32 裸机的 NVIC）

中断触发方式：电平触发（高 / 低电平）、边沿触发（上升 / 下降 / 双边沿）

中断优先级：内核通过 `irq_set_priority` 设置，ARM 架构中 GIC 的优先级有 16/256 级，数值越小优先级越高

2.驱动中断开发

(1) 获取设备号

设备树中定义中断属性，驱动中解析中断号

静态指定，仅调试用

(2) 注册中断处理函数 request_irq

会激活(使能)中断，可能会导致睡眠，不能在中断上下文或者其他禁止睡眠的代码段中使用

编写中断处理函数 `irqreturn_t (*irq_handler_t) (int, void *)`

`return IRQ_HANDLED;`//中断已处理

(3) 卸载中断

`free_irq(irq, dev);` // 释放中断，`dev_id`必须和`request_irq`一致，删除中断处理函数并且禁止中断

`tasklet_kill(&dev->tasklet);` // 销毁底半部

3.上半部：中断处理函数，**快速执行，禁止阻塞**（不能调用 `msleep`、`mutex_lock` 等）、**禁止耗时操作**

清中断标志、检查中断源、触发底半部、返回中断状态

4.下半部机制：中断处理函数中比较耗时的代码

(1) 软中断 `softirq_action`：网络 / 块设备等核心子系统（驱动不用）

(2) **tasklet**：短耗时、无阻塞的延迟处理（如串口数据拷贝）

定义tasklet和处理函数

使用`tasklet_init`函数初始化tasklet

（也可以使用宏`DECLARE_TASKLET`来一次性完成tasklet的定义和初始化）

上半部触发tasklet处理函数：在中断处理函数中调用`tasklet_schedule`函数

(3) 工作队列：长耗时、需阻塞的处理（如数据解析、上报）

在进程上下文执行，工作队列将要推后的工作交给一个内核线程去执行，因为工作队列工作在进程上下文，因此工作队列允许睡眠或重新调度

7. 内核定时器

1.Linux内核定时器采用系统时钟实现，需要提供**超时时间**和**定时处理函数**

2.API

定义 `timer_list` 表示定时器，`expires`表示超时时间（节拍数），`function`时定时处理函数，周期为2秒，超时时间为 `jiffies+(2*HZ)`

初始化 `init_timer`

注册 `add_timer`

删除 `del_timer` (`del_timer_sync` 等待其他处理器使用完后删除，不能在中断上下文使用)

修改定时值并激活定时器 `mod_timer`

`mod_timer(&dev->timertest, jiffies + msecs_to_jiffies(2000));` //重新设置超时值并且启动定时器

8. 按键驱动

设备结构体

key_device设备结构体

```
1  struct imx6uirq_dev{
2      dev_t devid;
3      struct cdev cdev;
4      struct class *class;
5      struct device *device;
6      struct device_node nd;
7      struct irq_keydesc{
8          int gpio;                //io编号
9          int irqnum;              //中断号
10         unsigned char value;     //键值
11         char name;               //名字
12         irqreturn_t (*handler) (int,void *);    //中断处理函数
13     };
14     struct timer_list timer;     //内核定时器
15
16     atomic_t keyvalue;
17     atomic_t keyrelease;
18
19     wait_queue_head_t r_wait;    //读等待队列头
20 };
```

1.修改设备树添加按键的platform设备节点和pinctrl节点

2.在模块入口函数注册platform驱动：key_driver（platform_driver_register），最终会调用key_probe

key_driver

```
1  struct platform_driver key_driver={
2      .driver={
3          .name="imx6ull-key",    /* 无设备树和设备进行匹配, 驱动名字 */
4          .of_match_table=key_of_match, /* 设备树匹配表 */
5      },
6      .probe=key_probe,
7      .remove=key_remove,
8  };
```

3.在驱动的probe函数中注册字符设备结构：key_probe

(1) 自动分配设备号：alloc_chrdev_region(设备号,从设备号分配起始,设备数量,设备名字);

(2) 初始化cdev：cdev_init(cdev结构体,设备操作函数集);

(3) 注册cdev: cdev_add(cdev结构体,设备号,设备数量);

(4) 创建类和设备: key_device.class=class_create(THIS_MODULE,设备名字);

key_device.device=device_create(类,NULL,设备号,NULL,设备名字);

4.构建设备操作函数集 file_operations

(1) open中按键、中断初始化

获取设备节点->申请、初始化IO->初始化中断->初始化定时器

从platform驱动中获取设备节点: key_device.nd=key_driver->dev.of_node

获取IO编号: of_get_named_gpio(设备节点, "key-gpio", 序号)

申请IO: gpio_request(IO编号, IO名称)

设置为输入: gpio_direction_input(IO编号)

获取中断号: gpio_to_irq(IO编号)

初始化并触发中断: request_irq(中断号, 中断服务函数, 触发方式, IO设备, 参数)

定义定时器: init_timer并设置function(定时器结构体), 设置定时服务函数

(2) read中实现阻塞上报

定义队列项->将队列项添加到等待队列头, 设置当前进程为可打断状态->等待wakeup唤醒->唤醒后设置当前进程为运行状态, 移除等待队列头

定义一个等待队列项: DECLARE_WAITQUEUE(wait,current)

将队列项添加到等待队列头: add_wait_queue(等待队列头,&wait)

当前进程设置为可被打断的状态: __set_current_state(TASK_INTERRUPTIBLE)

切换: schedule()

等待唤醒: 1.按键有效wakeup唤醒 2.被信号唤醒 signal_pending(current)

将当前任务或线程设置为运行状态:_set_current_state(TASK_RUNNING)

将对应的队列项从等待队列头删除:remove_wait_queue(等待队列头,&wait)

5.中断服务函数

(1) 按键中断处理函数

触发10ms定时器超时: mod_timer(&dev->timer,jiffies+msecs_to_jiffies(10))

(2) 定时服务函数 (消抖)

修改键值

唤醒进程 wake_up(等待队列头)

6.在模块出口函数注销platform驱动: key_remove (platform_driver_unregister)

platform_driver_unregister自动执行key_remove

释放gpio和中断：free_irq(中断号,&imx6irq)

删除定时器：del_timer_sync(定时器结构体)

删除cdev：cdev_del (cdev结构体)

注销设备号：unregister_chrdev_region(设备号,设备数量)

删除类和设备

9. MMU内存管理单元

给 Linux / Android / Win 等复杂系统提供虚拟内存支持，用于 Cortex-A 系列

1.虚拟地址 ↔ 物理地址

2.内存保护（权限、特权级、执行权限）

3.内存映射（设备、Flash、RAM 统一映射）

驱动里的 ioremap 本质是调用 MMU 映射物理地址到虚拟地址

4.按需分页、swap、进程地址空间隔离（每个进程有独立虚拟地址空间）